Protocole de Calibrage de Caméra pour Picamera (OpenCV)

Modèle Gemini

13 novembre 2025

Résumé

Ce document détaille le protocole standard de calibrage de caméra (Camera Calibration) utilisant un motif d'échiquier et la librairie OpenCV. L'objectif est de déterminer la matrice intrinsèque de la caméra et ses coefficients de distorsion pour une Picamera.

1 Objectif et Principe

Le calibrage de caméra est la procédure visant à déterminer la relation mathématique entre les coordonnées 3D d'un point dans le monde réel et ses coordonnées 2D projetées sur le capteur de la caméra. Les deux résultats principaux sont la **Matrice de Caméra** (camera_matrix) et les **Coefficients de Distorsion** (dists_coeff).

2 1. Préparation du Matériel et des Logiciels

2.1 A. Matériel Requis

— **Caméra :** La Picamera (accès via `picamera2`).
— **Échiquier :** Un motif imprimé plat. Vous devez connaître précisément le nombre de **coins intérieurs** ($N_W \times N_H$).
— **Support :** Fixation stable pour la caméra et conditions d'éclairage uniformes.

2.2 B. Logiciels Requis

Installation des dépendances Python (sur le Raspberry Pi) :

```
pip install picamera2 opencv-python numpy
```

3 2. Acquisition des Images ($\approx 15 - 20$ images)

L'acquisition d'images est l'étape la plus critique. Il faut obtenir des images de l'échiquier sous diverses perspectives pour couvrir tout le champ de vision et la distorsion de l'objectif.

3.1 Protocole de Capture

1. Capturer l'échiquier sous différents angles et distances.
2. Inclure des images où l'échiquier est positionné dans les quatre coins du champ de vision.
3. Assurer une netteté et un contraste suffisants pour la détection des coins par OpenCV.

4 3. Calibrage et Calculs (OpenCV)

4.1 A. Variables de Calibrage

— **Object Points (objp) :** Coordonnées 3D réelles des coins (dans un référentiel où $Z = 0$).
— **Image Points (imgpoints) :** Coordonnées 2D (en pixels) des coins détectés dans chaque image.

4.2 B. Détection des Coins par Image

Pour chaque image capturée (I) :

1. Conversion en niveaux de gris (gray).
2. **Détection :** Utilisation de `cv2.findChessboardCorners(gray, (N_W, N_H), None)`.
3. **Raffinage :** Si la détection réussit, utilisation de `cv2.cornerSubPix()` pour affiner la position des coins à une précision sous-pixel.
4. Stockage des paires (objp, imgpoints).

4.3 C. Fonction Principale de Calibrage

La fonction `cv2.calibrateCamera` effectue le calcul final :

```
ret, camera_matrix, dists_coeff, rvecs, tvecs = \
    cv2.calibrateCamera(objpoints, imgpoints, img_size, None, None)
```

4.3.1 Matrice de Caméra (camera_matrix)

Elle est de taille 3×3 et contient les paramètres intrinsèques (longueurs focales et point principal) :

$$\text{camera_matrix} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$$

— f_x, f_y : Longueurs focales en pixels.
— c_x, c_y : Coordonnées du Point Principal (centre optique).

4.3.2 Coefficients de Distorsion (dists_coeff)

C'est un vecteur qui modélise la distorsion de la lentille :

$$\text{dists_coeff} = (k_1, k_2, p_1, p_2, k_3)$$

— k_1, k_2, k_3 : Composantes de la **distorsion radiale**.
— p_1, p_2 : Composantes de la **distorsion tangentielle**.

Ces deux résultats sont les paramètres finaux nécessaires pour la correction d'image et la vision par ordinateur métrique.